

# **FPGA VOXGUARD**

## **Project Phase II Report**

Submitted by

**SREEJITH T.R (FIT22EC119)**

**SHARON BENNY (FIT22EC115)**

**MOHAMED AFSAL (FIT22EC093)**

**PRANAV RAMESH MANNATT(FIT22EC107)**

*In partial fulfillment of the requirements for award of the degree of*

**Bachelor of Technology**

in

**Electronics & Communication Engineering**



*Focus on Excellence*

**FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY (FISAT)<sup>®</sup>**

Angamaly-683577, Ernakulam

Affiliated to

**APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY**

Thiruvananthapuram-695016

**November 2025**

# FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY (FISAT)<sup>®</sup>

Mookkannoor(P.O), Angamaly-683577



*Focus on Excellence*

## CERTIFICATE

This is to certify that the project phase I report titled **FPGA VOXGUARD** submitted by **PRANAV RAMESH MANNATT (FIT22EC107)**, **MOHAMED AFSAL (FIT22EC093)**, **SREEJITH T.R (FIT22EC119)**, and **SHARON BENNY (FIT22EC115)** towards partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Electronics and Communication Engineering** is a record of bonafide work carried out by them during the academic year **2025–2026**.

**Project Guide**

**Project Coordinator**

**Head of the Department**

**Ms. Subha Thomas**

**Ms. Aiswariya Raj**

**Dr. Bejoy Varghese**

**Internal Examiner**

**External Examiner**

Place:

Date:

# Acknowledgement

We would like to express our profound gratitude to all the individuals who have supported and guided us throughout the journey of this project. Their invaluable contributions have been instrumental in the successful completion of our work.

We are extremely thankful to **Dr. Jacob Thomas V**, Principal, for providing us with the opportunity and the institutional support required to carry out this project successfully

We are extremely thankful to **Dr. BEJOY VARGHESE**, Associate Professor and Head of the Department of Electronics & Communication Engineering, for his support and for fostering an environment conducive to research and innovation. His leadership has enabled us to access the necessary resources and facilities required for our project work.

We extend our deepest and most sincere gratitude to our project guide, **Ms. SUBHA THOMAS**, Assistant Professor (Senior Grade), Department of Electronics & Communication Engineering. Her constant encouragement, insightful feedback, and expert technical guidance were the driving force behind this project. She navigated us through every challenge, from theoretical conception to hardware design, and her mentorship has been an invaluable learning experience for which we are truly grateful.

Our special thanks go to our Project Coordinator, **Ms. AISWARIYA RAJ**, Assistant Professor (Senior Grade), Department of Electronics & Communication Engineering, for her meticulous coordination and for ensuring that the project progressed smoothly according to the defined timelines. Her administrative support and guidance have been crucial in keeping our work on track.

Finally, we would also like to thank the other faculty members of the department for their helpful suggestions, as well as our friends and family for their unwavering support and encouragement throughout this endeavor.

# Abstract

*As the need for secure communication grows, traditional wireless voice systems remain vulnerable to eavesdropping. This project, "FPGA VoxGuard," details the design and simulation of a secure digital voice system that uses chaos theory for robust, hardware-friendly encryption. At its core is a stream cipher that encrypts a digitized audio signal by XORing it with a chaotic keystream generated by a logistic map.*

*Implemented on a Field Programmable Gate Array (FPGA) using Verilog, Voxguard leverages the highly nonlinear properties of chaotic systems to secure digital audio streams. The cryptographic core utilizes discretized chaotic algorithms—specifically, the Logistic map and the Skew Tent map—to generate unpredictable, pseudo-random key sequences. By exploiting the extreme sensitivity to initial conditions inherent in these chaotic maps, the system achieves strong cryptographic confusion and diffusion. The hardware-centric approach on an FPGA ensures highly parallelized processing, making Voxguard exceptionally well-suited for real-time secure communication applications where software-based encryption might introduce unacceptable latency. This report details the architectural design, Verilog implementation, and functional verification of the Voxguard system.*

# CONTENTS

## LIST OF FIGURES

## LIST OF TABLES

|                  |                                                        |          |
|------------------|--------------------------------------------------------|----------|
| <b>Chapter 1</b> | <b>INTRODUCTION</b>                                    | <b>1</b> |
| 1.1              | BACKGROUND AND MOTIVATION                              | 1        |
| 1.2              | INTRODUCTION TO CHAOS THEORY                           | 1        |
| 1.3              | PROBLEM STATEMENT                                      | 2        |
| 1.4              | PROJECT OBJECTIVES                                     | 2        |
| 1.5              | SCOPE OF THE PROJECT                                   | 2        |
| 1.6              | RELEVANCE AND SDG ALIGNMENT                            | 2        |
| 1.7              | STRUCTURE OF THE REPORT                                | 3        |
| <b>Chapter 2</b> | <b>LITERATURE REVIEW</b>                               | <b>4</b> |
| 2.1              | THE LANDSCAPE OF CHAOS-BASED CRYPTOGRAPHY              | 4        |
| 2.2              | FPGA IMPLEMENTATION FOR HIGH-THROUGHPUT SYSTEMS        | 4        |
| 2.3              | ADVANCEMENTS IN CHAOTIC MAP DESIGN AND SECURITY        | 5        |
| 2.4              | SYNCHRONIZATION AND PRACTICAL COMMUNICATION CHALLENGES | 5        |
| 2.5              | FRAMEWORKS FOR SECURITY AND PERFORMANCE EVALUATION     | 6        |
| 2.6              | CONCLUSION AND RESEARCH POSITION                       | 6        |
| <b>Chapter 3</b> | <b>DESIGN AND IMPLEMENTATION</b>                       | <b>8</b> |
| 3.1              | SYSTEM ARCHITECTURE                                    | 8        |
| 3.1.1            | Two-Phase Methodology                                  | 8        |
| 3.1.2            | High-Level System Block Diagram                        | 8        |
| 3.2              | MATLAB SIMULATION                                      | 8        |
| 3.2.1            | The Chaotic Keystream Generator: Skew Tent Map         | 8        |
| 3.2.2            | Encryption and Decryption Process                      | 9        |
| 3.3              | SYSTEMVERILOG IMPLEMENTATION                           | 10       |
| 3.3.1            | Top-Level Design and Data Flow                         | 10       |
| 3.3.2            | Chaotic Keystream Generator Module                     | 10       |
| 3.3.3            | Encryption and Peripheral Modules                      | 10       |
| 3.4              | SUMMARY                                                | 11       |

|                  |                                                                    |           |
|------------------|--------------------------------------------------------------------|-----------|
| <b>Chapter 4</b> | <b>HARDWARE AND SOFTWARE</b>                                       | <b>12</b> |
| 4.1              | Software and Logic Implementation                                  | 12        |
| 4.1.1            | I2S Protocol and Digital Audio Processing                          | 12        |
| 4.1.2            | Hybrid Chaotic Key Generation Logic                                | 12        |
| 4.1.3            | SPI Communication and nRF24L01+ Driver                             | 12        |
| 4.1.4            | Central Control Finite State Machine (FSM)                         | 13        |
| 4.2              | Hardware Architecture and Implementation                           | 13        |
| 4.2.1            | Digilent Cmod S7 FPGA Development Board                            | 13        |
| 4.2.2            | Audio Acquisition: INMP441 I2S Microphone                          | 13        |
| 4.2.3            | Wireless Transmission: nRF24L01+ Radio Transceiver                 | 14        |
| 4.2.4            | Audio Output: PCM5102A I2S DAC and PAM8403 Amplifier               | 14        |
| <b>Chapter 5</b> | <b>RESULTS AND DISCUSSIONS</b>                                     | <b>15</b> |
| 5.1              | SIMULATION RESULTS: ENCRYPTION & DECRYPTION                        | 15        |
| 5.2              | EXTENDED ANALYSIS: REAL-WORLD CHALLENGES                           | 15        |
| 5.3              | Digital Core and Cryptographic Pipeline Verification               | 17        |
| 5.4              | Analog Audio Output Diagnostics                                    | 17        |
| 5.5              | System Output                                                      | 17        |
| <b>Chapter 6</b> | <b>CONCLUSION</b>                                                  | <b>19</b> |
| 6.1              | SUMMARY OF WORK                                                    | 19        |
| 6.2              | KEY FINDINGS                                                       | 19        |
| 6.3              | PROJECT OUTCOMES                                                   | 19        |
| 6.4              | FUTURE SCOPE                                                       | 20        |
| 6.5              | CONCLUDING REMARKS                                                 | 20        |
|                  | <b>REFERENCES</b>                                                  | <b>22</b> |
| <b>Chapter A</b> | <b>PROJECT CODE LISTINGS</b>                                       | <b>23</b> |
| A.1              | MATLAB SIMULATION SCRIPTS                                          | 23        |
| A.1.1            | Keystream Generation Script                                        | 23        |
| A.1.2            | Encryption and Decryption Core Script                              | 23        |
| A.1.3            | Complete MATLAB Simulation Script                                  | 24        |
| A.2              | SYSTEMVERILOG HARDWARE DESCRIPTION LANGUAGE (HDL) MODULES          | 28        |
| A.2.1            | Top-Level Module ( <code>top_level.sv</code> )                     | 28        |
| A.2.2            | Chaotic Keystream Generator ( <code>chaotic_generator.sv</code> )  | 31        |
| A.2.3            | Packet Manager FSM ( <code>packet_manager.sv</code> )              | 32        |
| A.2.4            | Encryption and Decryption Core ( <code>encrypt_decrypt.sv</code> ) | 35        |
| A.2.5            | I2S Controller ( <code>i2s_controller.sv</code> )                  | 36        |



# List of Figures

|     |                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------|----|
| 3.1 | High-Level Block Diagram of the Proposed Hardware System                                   | 9  |
| 3.2 | Block Diagram of the MATLAB Simulation Process                                             | 9  |
| 3.3 | Diagram of RTL blocks within the FPGA                                                      | 10 |
| 5.1 | Waveforms of the (Top) Original, (Middle) Encrypted, and (Bottom) Decrypted Audio Signals. | 16 |
| 5.2 | Waveform of Failed Decryption due to Key Sensitivity.                                      | 16 |
| 5.3 | Waveform of Failed Decryption due to Channel Noise (25dB SNR).                             | 16 |
| 5.4 | Waveform of Failed Decryption due to Synchronization Error.                                | 16 |

# List of Tables

|                               |
|-------------------------------|
| 2.1 Literature Review Summary |
|-------------------------------|

# Chapter 1

## INTRODUCTION

### 1.1 BACKGROUND AND MOTIVATION

The proliferation of wireless communication technologies has fundamentally reshaped modern society, enabling unprecedented connectivity across a vast array of devices and platforms. From personal communication and Internet of Things (IoT) ecosystems to critical industrial and military applications, the reliance on wireless infrastructure continues to grow. However, this convenience comes with an inherent vulnerability: the broadcast nature of the wireless medium makes it susceptible to unauthorized interception. Traditional analog voice systems, such as conventional walkie-talkies and radios, offer minimal to no data privacy, with modulation schemes that can be easily demodulated by third-party receivers. While many modern digital systems incorporate encryption, they often rely on standardized, computationally intensive block ciphers or, in low-cost devices, weak and outdated security protocols. This creates a significant gap in the market for communication systems that are simultaneously secure, affordable, and suitable for real-time applications like voice transmission. The growing concerns over data privacy in both civilian and industrial settings demand novel solutions. There is a clear need for a secure communication paradigm that is not only robust against sophisticated attacks but also efficient enough to be implemented on resource-constrained hardware platforms. This project, "FPGA VoxGuard," is motivated by this need, aiming to develop a next-generation solution for secure, real-time wireless audio communication by leveraging the powerful and inherently secure properties of chaos theory.

### 1.2 INTRODUCTION TO CHAOS THEORY

Chaos theory is a branch of mathematics focused on dynamical systems whose seemingly random behavior is actually governed by deterministic laws. This fascinating duality makes it highly applicable to cryptography. The three key properties of chaos are central to this project. First is determinism, which ensures that a chaotic system will always produce the same output sequence if given the exact same initial conditions. This allows a receiver with a secret key (the initial conditions) to perfectly replicate the keystream needed for decryption. Second is the system's unpredictable and random-like behavior, where the output is aperiodic and passes most statistical tests for randomness, making it an ideal source for a cryptographic keystream. Finally, and most famously, is the extreme sensitivity to initial conditions, often called the "Butterfly Effect." This property guarantees that a microscopic change in the starting key will cause the system's output to diverge wildly, rendering any decryption attempt with a slightly incorrect key completely useless. These natural properties make chaotic systems a lightweight

yet powerful alternative to traditional pseudo-random number generators.

### **1.3 PROBLEM STATEMENT**

Current analog and even most low-cost digital wireless voice systems offer poor data privacy, with simple modulation schemes or weak encryption. This makes them prone to eavesdropping and replay attacks. Chaos-based cryptography offers a promising alternative by using the highly sensitive and non-repetitive character of chaotic systems, such as the Skew Tent map, to create strong, hardware-friendly encryption schemes. The primary challenge lies in the efficient realization of these algorithms on readily available hardware platforms like FPGAs to yield an affordable, custom, and highly secure walkie-talkie replacement that can operate reliably in real-world conditions.

### **1.4 PROJECT OBJECTIVES**

The primary objectives of this project are to develop a chaotic map-based encryption algorithm and to design a complete communication system for its implementation on an FPGA platform. This involves creating a system specifically tailored for the secure encryption of audio and speech signals, ensuring that the final design is computationally efficient enough to achieve real-time processing capabilities suitable for live voice communication.

As secondary objectives, we aim to validate the security of the proposed system through comprehensive statistical analysis within a simulated environment. This simulation will also be used to identify and quantify the primary challenges of a physical implementation, namely sensitivity to channel noise and the need for perfect synchronization. Finally, the hardware design will be optimized for efficient resource utilization, and the entire project will serve to demonstrate the practical application of chaos theory in a functional proof-of-concept.

### **1.5 SCOPE OF THE PROJECT**

The scope of this project encompasses the design and simulation of a chaos-based encryption algorithm utilizing the skew Tentt map, specifically applied to audio files. A comprehensive security and performance analysis will be conducted based on the simulation results. Furthermore, this project includes the complete hardware design of all required digital logic modules in SystemVerilog, creating a full blueprint for an FPGA-based implementation.

However, the project's scope does not extend to the implementation of quantum cryptography, the development of novel network protocols, or the final stages of commercial product development and manufacturing. Additionally, the implementation will be purely digital, and the design of analog circuits is not covered.

### **1.6 RELEVANCE AND SDG ALIGNMENT**

This project directly contributes to the UN's 2030 Agenda for Sustainable Development, particularly by fostering innovation and building resilient technological infrastructure, aligning with Sustainable Development Goal 9 (Industry, Innovation, and Infrastructure). By exploring

a novel method of secure communication using chaos theory, we are pushing beyond traditional algorithms. The project provides a blueprint for creating low-cost, resilient, and independent secure communication hardware. As a research endeavor, it enhances scientific knowledge in applied chaos theory and builds practical skills in advanced FPGA design.

The project also supports Sustainable Development Goal 16 (Peace, Justice, and Strong Institutions). Secure, private communication is a fundamental tool for protecting freedoms and strengthening institutions. The technology developed in this project can help ensure the confidentiality of communications for journalists, researchers, and organizations, protecting them from unauthorized surveillance and contributing to a more just and secure society.

## **1.7 STRUCTURE OF THE REPORT**

This report is organized into seven chapters. *Chapter 1* provides an introduction to the project, its motivation, and objectives. *Chapter 2* presents a comprehensive review of the existing literature in the field of chaotic communications and FPGA-based encryption. *Chapter 3* details the complete design of the system, covering the MATLAB simulation of the communication process. *Chapter 4* outlines the estimated cost of building the physical prototype. *Chapter 5* presents the results and a detailed discussion of the MATLAB simulation, including the security analysis and tests for real-world challenges. *Chapter 6* provides a comparative analysis of our system's performance against metrics from the literature and traditional methods. Finally, *Chapter 7* concludes the report with a summary of the findings and a discussion of future work.

## Chapter 2

### LITERATURE REVIEW

#### 2.1 THE LANDSCAPE OF CHAOS-BASED CRYPTOGRAPHY

The field of chaos-based cryptography has emerged as a compelling alternative to conventional cryptographic methods, leveraging the inherent properties of chaotic systems for secure communications. The deterministic yet unpredictable nature of these systems, alongside their extreme sensitivity to initial conditions, makes them ideal for generating high-quality pseudo-random number sequences <sup>[1]</sup>. A dominant trend in the literature is the implementation of these complex mathematical systems on hardware platforms, particularly Field-Programmable Gate Arrays (FPGAs). This approach is driven by the need for high throughput and real-time performance, which are critical for securing modern multimedia data streams such as audio and video <sup>[2, 3]</sup>.

However, a critical review of the literature reveals a persistent gap between theoretical simulations and validated, end-to-end hardware prototypes. While many novel chaotic systems are proposed, fewer studies present complete implementations that address practical challenges like audio interfacing, robust synchronization, and resilience to channel noise. This project is positioned to bridge this gap, building upon the foundational work in the field to develop a practical and robust hardware solution for real-time audio encryption.

#### 2.2 FPGA IMPLEMENTATION FOR HIGH-THROUGHPUT SYSTEMS

The use of FPGAs is central to achieving the performance required for real-time chaotic encryption. The literature provides strong evidence of the platform's suitability, offering valuable benchmarks and design roadmaps. For instance, F. Wang et al. successfully implemented a novel multi-shape chaotic attractor on a Cyclone IV FPGA, reaching an operating frequency of 117.4 MHz, which serves as a practical performance benchmark for similar hardware <sup>[4]</sup>.

Efficiency in hardware is often achieved through the simplification of the chaotic system itself. H. Lin et al. demonstrate this by designing a system that avoids complex polynomial functions, thereby reducing resource requirements. Their implementation achieved an exceptional throughput of 3.39 Gbps for image encryption, validating the choice of FPGAs for high-speed applications <sup>[2]</sup>. Furthermore, the work of I. Damaj et al. provides a crucial analysis of the trade-offs between precision, throughput, and power in FPGA designs. Their research into high-precision Lorenz chaos cores, achieving speeds up to 21 Gbps, directly informed this project's decision to use 32-bit fixed-point arithmetic, striking a balance between numerical accuracy and resource efficiency <sup>[3]</sup>. These studies collectively confirm that FPGAs are not only capable but are the preferred platform for implementing high-performance chaotic cryptosys-

tems.

### **2.3 ADVANCEMENTS IN CHAOTIC MAP DESIGN AND SECURITY**

The security of any chaos-based cryptosystem is fundamentally dependent on the statistical quality of its underlying chaotic map. The classical logistic map, while foundational, is known to have weaknesses. Addressing this, D. Herbadji et al. developed an enhanced logistic map with a wider chaotic range and superior statistical uniformity (ergodicity). Its robustness was validated using the rigorous NIST SP 800-22 test suite, demonstrating a clear path for improving the statistical foundation of chaotic generators <sup>[5]</sup>.

Another avenue for enhancing security is to increase the complexity of the chaotic system. T. Haridas et al. explored the use of higher-dimensional (2D/3D) hyperchaotic systems for audio encryption <sup>[6]</sup>. These systems are inherently more unpredictable than simple 1D maps, offering a significantly larger keyspace and greater resistance to attacks while preserving audio quality. This research highlights a clear and promising direction for future work, suggesting that the security of a functional prototype could be substantially upgraded by integrating a more complex, higher-dimensional chaotic map.

### **2.4 SYNCHRONIZATION AND PRACTICAL COMMUNICATION CHALLENGES**

A persistent and critical challenge in chaotic communication is achieving and maintaining synchronization between the transmitter and receiver. Hwang et al. approached this problem from a modern, simulation-based perspective, applying machine learning algorithms to improve synchronization accuracy in a system based on Chua's circuit. Their work, while confined to MATLAB Simulink, suggests that ML techniques could offer a novel solution to this long-standing issue <sup>[7]</sup>.

In contrast, other researchers have focused on complete hardware implementations to solve real-world communication problems. Z. M. Alroubaie et al. detailed a full FPGA-based speech encryption system using a master-slave synchronization scheme, validated on a Xilinx SP605 board. Their work provides a valuable, practical comparison for hardware-based synchronization protocols, though they note the absence of a secure key exchange mechanism—a gap this project addresses <sup>[8]</sup>. Similarly, C. Bai et al. presented a complete chaotic radio-wave communication system, innovating with a Chaotic Shape-forming Filter (CSF) that demonstrated a superior Bit Error Rate (BER) in multipath channels compared to traditional filters <sup>[9]</sup>. These studies underscore the importance of moving beyond simulation to tackle the practical challenges of synchronization and channel noise in physical communication systems.

## 2.5 FRAMEWORKS FOR SECURITY AND PERFORMANCE EVALUATION

Rigorous and standardized evaluation is necessary to meaningfully assess the security of a chaos-based encryption scheme. The literature provides several frameworks and benchmarks for this purpose. The comprehensive review by R. B. Naik et al. serves as a critical resource, compiling key performance statistics from a wide range of chaotic maps <sup>[1]</sup>. It provides essential benchmarks, such as a lowest correlation coefficient of 0.00006 and a highest entropy of 7.999995 bits/byte, which allow for the contextualization of any new system's performance within the broader research field.

While general randomness tests are vital, specific metrics are required to evaluate performance for particular data types. For audio encryption, X. Zhou et al. established a quantitative framework, providing ideal target values for key security metrics. These include a Number of Samples Change Rate (NSCR) greater than 99.99% and a Unified Average Changing Intensity (UACI) of approximately 33.33% <sup>[10]</sup>. Together, these papers provide a comprehensive toolkit for rigorously validating the security and performance of an audio encryption system, addressing the inconsistent evaluation approaches often seen across different studies.

## 2.6 CONCLUSION AND RESEARCH POSITION

The body of literature provides a robust foundation for chaos-based cryptography, confirming the viability of using FPGAs for high-throughput, real-time applications. Key research themes consistently emerge: the pursuit of hardware efficiency, the enhancement of chaotic map security, and the ongoing challenge of synchronization in practical communication systems. While significant advancements have been made in theoretical design and simulation, a notable gap persists between these models and complete, end-to-end hardware implementations that are validated in real-world conditions.

This project is strategically positioned to address this gap. By focusing on the development of a complete hardware prototype—from audio interfacing to radio transmission—it moves beyond the simulation-heavy focus of much of the literature. The project directly confronts the practical challenges of robust synchronization and resilience to channel noise, areas that are critical for deployment but are often secondary in theoretical studies. Furthermore, by employing a comprehensive suite of standardized security and performance metrics, this work aims to provide the clear, rigorous, and consistent validation that is needed to bridge the gap between theoretical potential and demonstrated, practical security.

Table 2.1: Literature Review Summary

| No. | Title & Author(s)                                                                   | Key Contributions/ Insights                                                                                                                               | Performance Metrics                                 | Identified Gaps/ Relevance                            |
|-----|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| 1   | "Enhancing Continuous Chaos Communication...", Hwang et al.                         | <ul style="list-style-type: none"> <li>• Uses ML to improve synchronization.</li> <li>• MATLAB Simulink implementation.</li> </ul>                        | Sync accuracy improved from 86%.                    | <b>Gap:</b> Lacks hardware validation.                |
| 2   | "Radio-Wave Communication With Chaos", C. Bai et al.                                | <ul style="list-style-type: none"> <li>• Proposes CSF for baseband signals.</li> <li>• Validated with hardware.</li> </ul>                                | Superior BER in multipath channels.                 | <b>Gap:</b> Lacks multimedia focus.                   |
| 3   | "Multi-Shape Chaotic Attractor FPGA...", F. Wang et al.                             | Generating $2n+2$ attractor types (scroll, star, flower).                                                                                                 | 117.4 MHz on Cyclone IV.                            | Hardware implementation roadmap.                      |
| 4   | "Multi wing Chaotic System for Encryption", H. Lin et al.                           | Simple system design; reduced component requirements.                                                                                                     | 3.39 Gbps for image encryption.                     | Hardware implementation roadmap.                      |
| 5   | "Enhanced Logistic Chaotic Map...", D. Herbadji et al.                              | Map with wider chaotic range and better ergodicity.                                                                                                       | Real-time processing capability.                    | N/A                                                   |
| 6   | "FPGA Speech Encryption using Synchronised Chaotic Maps", Z.M Alroubaie et al.      | <ul style="list-style-type: none"> <li>• Master-slave synchronization for five chaotic maps.</li> <li>• Validated on Xilinx SP605.</li> </ul>             | N/A                                                 | <b>Gap:</b> Lack of key exchange like Diffie-Hellman. |
| 7   | "Review on Chaotic Maps in Pseudo-Random Number Generators", R.B Naik et al.        | Literature review compiling key statistics from various maps.                                                                                             | Lowest Correlation: 0.00006; Max Entropy: 7.999995. | N/A                                                   |
| 8   | "Study of Multimedia Digital Audio Encryption", X. Zhou et al.                      | Analysis of performance metrics for audio encryption.                                                                                                     | PSNR: -0.348 dB to -7.645 dB; NSCR: 99.99%.         | N/A                                                   |
| 9   | "Optimizing FPGA implementation of high-precision chaotic systems", I. Damaj et al. | <ul style="list-style-type: none"> <li>• Trade-offs: precision vs. throughput vs. power.</li> <li>• Importance of solver choice (Euler, RK-4).</li> </ul> | Lorenz chaos cores up to 21 Gbps at 64-bit.         | N/A                                                   |
| 10  | "Chaos-based audio encryption: Efficacy of 2D and 3D systems", T. Haridas et al.    | Uses 2D/3D hyperchaotic maps for audio encryption.                                                                                                        | Resistance to statistical/differential attacks.     | N/A                                                   |

## Chapter 3

# DESIGN AND IMPLEMENTATION

### 3.1 SYSTEM ARCHITECTURE

#### 3.1.1 Two-Phase Methodology

The design and implementation of the Voxguard system follow a structured, two-phase methodology. This approach mitigates risk by ensuring the theoretical and logical soundness of the design in a flexible simulation environment before committing to a rigid hardware implementation. The initial phase involves building a complete model of the communication system in MATLAB, which allows for rapid prototyping, comprehensive analysis, and the identification of critical challenges such as noise sensitivity and synchronization. The second phase then takes the validated model and the insights gained from the simulation to create a physical hardware design in SystemVerilog, focusing on a real-time, high-performance circuit that incorporates solutions to the challenges identified in Phase 1.

#### 3.1.2 High-Level System Block Diagram

The overall architecture of the final Voxguard system is a digital radio transceiver where each unit can both transmit and receive, enabling two-way communication, as seen in Fig.3.1. The data flow begins with an analog audio signal from a microphone, which is digitized by an external ADC and sent to the transmitter FPGA. Inside the FPGA, a chaotic generator produces a keystream that is used to encrypt the audio data. This encrypted data is then passed to a wireless transmitter module for over-the-air transmission. A second receiver unit captures this signal, and its own FPGA, using the same secret key, generates a replica keystream to decrypt the data. Finally, the decrypted digital audio stream is converted back to an analog signal by an external DAC for playback.

### 3.2 MATLAB SIMULATION

The MATLAB simulation serves as the proof-of-concept for the entire system, modeling the core processes of key generation, encryption, and decryption in a controlled, ideal environment. The complete simulation script is provided in Listing 3 in Appendix A. The flow of the MATLAB simulation is shown in Fig. 3.2.

#### 3.2.1 The Chaotic Keystream Generator: Skew Tent Map

For the initial proof-of-concept simulation, the *Skew Tent map* was chosen as the chaotic keystream generator. As a well-known and mathematically simple chaotic system, it provides an excellent baseline for validating the core encryption architecture. The Skew Tent map is

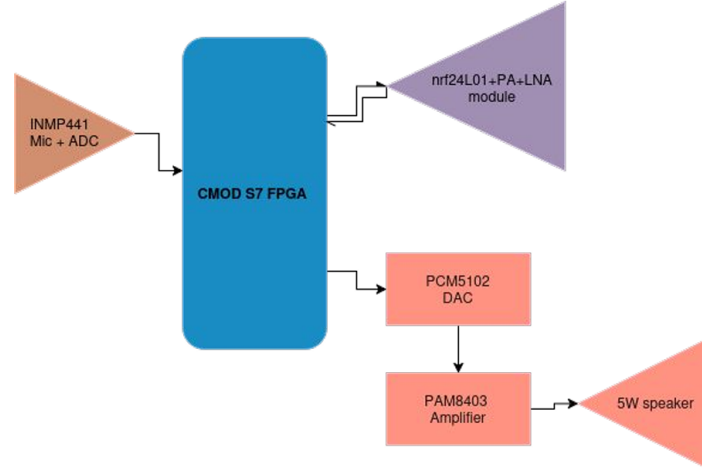


Figure 3.1: High-Level Block Diagram of the Proposed Hardware System

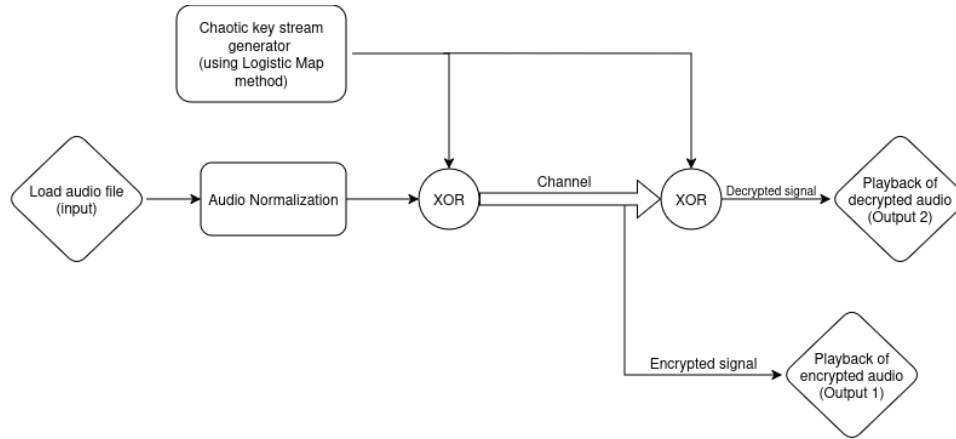


Figure 3.2: Block Diagram of the MATLAB Simulation Process

defined by the iterative equation  $x_{n+1} = r \cdot x_n \cdot (1 - x_n)$ . For this project, the system parameter  $r$  is set to 4.0, and the secret initial condition  $x(1)$  is set to 0.1234. The implementation of this iterative process to generate the key stream is shown in Listing 1 in the Appendix.

### 3.2.2 Encryption and Decryption Process

The Voxguard system employs a stream cipher architecture using a *bitwise XOR operation* (`bitxor` in MATLAB), which is highly efficient and its own inverse. Before this operation, the floating-point chaotic key (range  $[0.0, 1.0]$ ) and the audio signal are both converted to a signed 16-bit integer format. A scale and shift operation is used to map the key to the integer range  $[-2^{15}, +2^{15} - 1]$ , ensuring compatibility for the XOR operation. The core encryption and decryption logic is detailed in Listing 2 in the Appendix.

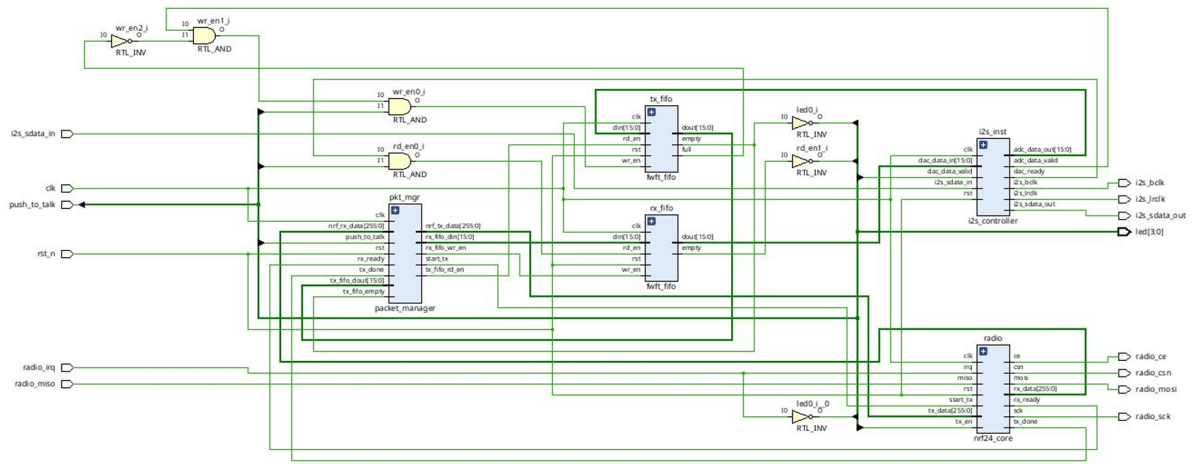


Figure 3.3: Diagram of RTL blocks within the FPGA

### 3.3 SYSTEMVERILOG IMPLEMENTATION

Following the successful validation in MATLAB, Phase 2 translates the model into a hardware implementation using SystemVerilog. This design addresses real-time processing and resource constraints, converting the floating-point chaotic map into a *fixed-point representation* suitable for efficient FPGA synthesis [3, 8]. The system utilizes a modular structure with distinct blocks for key generation, data management, encryption, and peripheral communication.

#### 3.3.1 Top-Level Design and Data Flow

The `top_level` module, provided in Listing 4, orchestrates the system. It instantiates and connects all other modules, defining the complete transceiver data path. The *Packet Manager* module, detailed in Listing 6, is the control center. Implemented as a Finite State Machine (FSM), it manages data buffering, enables the key generator, and controls the flow between transmit and receive modes based on the `push_to_talk` signal.

#### 3.3.2 Chaotic Keystream Generator Module

The hardware implementation of the Skew Tent map is contained within the `chaotic_generator` module. As shown in Listing 5, all calculations use 32-bit fixed-point numbers (Q2.30 format) to achieve high-precision results without dedicated floating-point hardware. The module includes a `sync_en` input to allow re-synchronization to an external state, a critical feature for maintaining key alignment in the receiver [9].

#### 3.3.3 Encryption and Peripheral Modules

The core encryption/decryption logic is implemented in the purely combinational `encrypt_decrypt` module, which performs a simple XOR operation between the 16-bit audio data and the upper 16 bits of the key, as shown in Listing 7 [6]. Peripheral interfacing is handled by dedicated modules: the *I2S Controller* (Listing 8) manages digital audio I/O with

the external codec, and the *SPI Master* (Listing 9) handles communication with the wireless radio module.

### **3.4 SUMMARY**

This chapter has detailed the comprehensive design process of the Voxguard system, following a structured two-phase methodology. The initial phase, conducted in MATLAB, successfully served as a proof-of-concept, validating the core cryptographic principles. The second phase translated this validated model into a practical and synthesizable hardware design using SystemVerilog. Key challenges, such as implementing the chaotic map's floating-point mathematics, were addressed by converting the algorithm to a high-precision fixed-point representation suitable for FPGA synthesis. With the complete Register-Transfer Level (RTL) design established, the project is now prepared for the crucial next stages of verification, synthesis, and physical implementation, which will be discussed in the following chapters.

## Chapter 4

# HARDWARE AND SOFTWARE

### 4.1 Software and Logic Implementation

The software architecture of the Voxguard system is defined primarily through Register Transfer Level (RTL) design using Verilog HDL. Unlike a traditional microprocessor-based approach, the "software" in this context refers to the hardware logic gates and state machines synthesized into the FPGA fabric. This concurrent execution model allows for the simultaneous processing of audio data, chaotic key generation, and wireless packet handling, ensuring that the encryption process introduces zero perceptible jitter or audio lag.

#### 4.1.1 I2S Protocol and Digital Audio Processing

The foundation of the digital datapath begins with the I2S Master Controller module. This module is responsible for generating the Bit Clock (BCLK) and Word Select (WS) signals required to synchronize with the INMP441 microphone and the PCM5102A DAC. The logic is designed to capture 24-bit audio samples by shifting serial data into an internal register on the rising edge of the BCLK. Once a full audio frame is captured, the module signals the encryption core that a new sample is ready for processing. This precise timing is critical to maintaining the audio sample rate—typically 44.1 kHz—without losing synchronization between the transmitter and receiver.

#### 4.1.2 Hybrid Chaotic Key Generation Logic

At the heart of the cryptographic layer is the Hybrid PRNG module, which implements the discretized Skew Tent Map and the Linear Feedback Shift Register (LFSR). The Skew Tent Map logic is implemented using fixed-point arithmetic, where the state variable is represented as a high-precision integer to minimize quantization errors. The piecewise-linear function is realized through a combinational comparator that selects between two arithmetic paths based on the control parameter. To ensure the chaotic sequence does not collapse into a short cycle due to the finite precision of the FPGA, a parallel LFSR module runs a continuous perturbation algorithm. At predefined intervals, the LFSR's output is XORed with the chaotic map's internal state, effectively "shaking" the system into a new trajectory and significantly increasing the overall period of the key stream.

#### 4.1.3 SPI Communication and nRF24L01+ Driver

The interface between the FPGA and the wireless transceiver is managed by a custom Serial Peripheral Interface (SPI) Master module. This module facilitates the exchange of commands and encrypted audio data with the nRF24L01+ radio. The software logic within this module

handles the complex power-on-reset sequences, channel configuration, and address matching required by the radio. Because the nRF24L01+ operates on a packet-based system rather than a continuous stream, a FIFO (First-In-First-Out) buffer is implemented in the FPGA fabric. This buffer temporarily stores encrypted audio samples until a full payload is ready for a burst transmission, ensuring that the wireless link remains efficient and robust against interference.

#### **4.1.4 Central Control Finite State Machine (FSM)**

The coordination of all sub-modules is governed by a central Control Finite State Machine (FSM) that monitors the system's operational mode. In its idle state, the FSM defaults to a "listening" mode, where the SPI module continuously polls the radio for incoming packets. When the user activates the Push-To-Talk (PTT) hardware button, the FSM transitions the system into "transmit" mode. During this transition, the FSM reconfigures the radio's internal registers, activates the chaotic key generator, and begins routing audio from the microphone to the encryption engine. This state-driven approach ensures that the system responds predictably to user inputs and maintains a stable communication link even in high-traffic RF environments.

## **4.2 Hardware Architecture and Implementation**

The physical realization of the Voxguard chaotic walkie-talkie system relies on a modular hardware architecture. The system is distributed across two identical transceiver units, each responsible for both the encryption of outgoing audio and the decryption of incoming streams. The hardware stack is centered around a Spartan-7 FPGA, augmented by dedicated audio and radio frequency (RF) peripherals.

### **4.2.1 Digilent Cmod S7 FPGA Development Board**

The core processing engine of the Voxguard unit is the Digilent Cmod S7 board, which houses a Xilinx Spartan-7 FPGA. This central component is responsible for executing the entire digital datapath. Its primary tasks include handling the I2S protocol interfacing, executing the LFSR and Skew Tent Map-based pseudo-random number generation, and driving the real-time XOR encryption and decryption engine. The Cmod S7 was selected specifically for its breadboard-friendly form factor and the powerful Digital Signal Processing (DSP) capabilities inherent to the Spartan-7 fabric. These DSP slices are particularly critical for efficiently computing the fixed-point multiplications required by the discretized Skew Tent Map, ensuring the system operates without consuming an excessive number of standard logic cells.

### **4.2.2 Audio Acquisition: INMP441 I2S Microphone**

To capture voice inputs with minimal latency and high fidelity, the system utilizes the INMP441, an omnidirectional MEMS microphone equipped with a built-in Inter-IC Sound (I2S) interface. This module captures analog sound waves and converts them directly into a digital Pulse Code Modulation (PCM) stream. By integrating a microphone with a native I2S digital output, the design effectively bypasses the need for an external Analog-to-Digital Converter (ADC).

This architectural choice streamlines the FPGA interface, requiring only three essential signal lines—Word Select, Serial Clock, and Serial Data—to feed raw audio directly into the hardware encryption pipeline.

#### **4.2.3 Wireless Transmission: nRF24L01+ Radio Transceiver**

The secure communication link between the two Voxguard units is established using the nRF24L01+ module, which operates in the 2.4 GHz ISM band. Once the FPGA XOR-encrypts the I2S audio stream with the chaotic key, the resulting ciphertext is packaged and sent over a Serial Peripheral Interface (SPI) to the nRF24L01+ for wireless transmission. On the receiving end, the matching nRF24L01+ captures these packets and forwards the encrypted payload back to its local FPGA for decryption. This specific transceiver is highly favored for embedded systems due to its low power consumption, built-in ShockBurst packet handling, and a sufficient data rate of up to 2 Mbps, which easily supports continuous voice streams.

#### **4.2.4 Audio Output: PCM5102A I2S DAC and PAM8403 Amplifier**

Reconstructing the decrypted digital audio into an audible signal requires a two-stage analog output block consisting of a DAC and an amplifier. The PCM5102A Digital-to-Analog Converter (DAC) is tasked with receiving the decrypted, plain-text I2S stream from the FPGA and converting it back into an analog line-level signal. Because this analog signal is generally too weak to drive a speaker directly, a PAM8403 Class-D audio amplifier is used in tandem. This highly efficient amplifier boosts the DAC's analog output to successfully drive the system's external speaker, completing the audio reconstruction phase.

## Chapter 5

# RESULTS AND DISCUSSIONS

This chapter presents the comprehensive results from the MATLAB simulation phase of the Voxguard project. The simulation was designed to validate the core concept of chaotic audio encryption and to identify the real-world challenges a physical hardware implementation must address, a common goal in modern communication system design <sup>[9]</sup>. Using a standard `.wav` audio file as the input signal, the following sections detail the results of the basic encryption/decryption cycle, the statistical security analysis, and an extended analysis of the system's robustness to practical imperfections.

### 5.1 SIMULATION RESULTS: ENCRYPTION & DECRYPTION

The initial objective was to confirm that the proposed encryption scheme is perfectly reversible under ideal, noiseless conditions. As shown in Figure 5.1, the original audio signal was encrypted using the chaotic keystream and then decrypted using the exact same key. The decrypted audio signal is visually identical to the original, while the encrypted signal appears as uniformly distributed noise with no discernible structure. This visual confirmation is supported by quantitative metrics, which reveal a Mean Squared Error (MSE) of zero and a perfect positive Correlation Coefficient of 1.0. These results mathematically confirm that there is zero difference between the original and recovered signals, successfully demonstrating that the core encryption/decryption cycle is lossless in an ideal environment.

### 5.2 EXTENDED ANALYSIS: REAL-WORLD CHALLENGES

A physical system will inevitably face imperfections, so three critical real-world failure scenarios were simulated. The first was a key sensitivity test, where decryption was attempted with a microscopically different key (0.123400000000001 vs 0.1234). The result was a catastrophic decryption failure (MSE: 0.305798, Correlation: -0.0012), as seen in Figure 5.2, confirming the system's high sensitivity to the secret key, a fundamental property of secure chaotic systems. The second test evaluated robustness to channel noise by adding Additive White Gaussian Noise (AWGN) at a 25dB Signal-to-Noise Ratio (SNR) to the encrypted signal. Even at this relatively good SNR, the decrypted audio was heavily distorted (MSE: 0.032152, Correlation: 0.5016), as shown in Figure 5.3. This highlights the system's high sensitivity to transmission errors and establishes the critical need for Forward Error Correction (FEC) logic in the physical hardware design, a common challenge in chaotic communication <sup>[7]</sup>.

Finally, a synchronization error test simulated a tiny mismatch in the chaos parameter  $r$  between the transmitter ( $r = 4.0$ ) and the receiver ( $r = 4.0 - 1 \times 10^{-12}$ ). This minuscule deviation resulted in complete decryption failure (MSE: 0.307349, Correlation: -0.0014), illustrated in

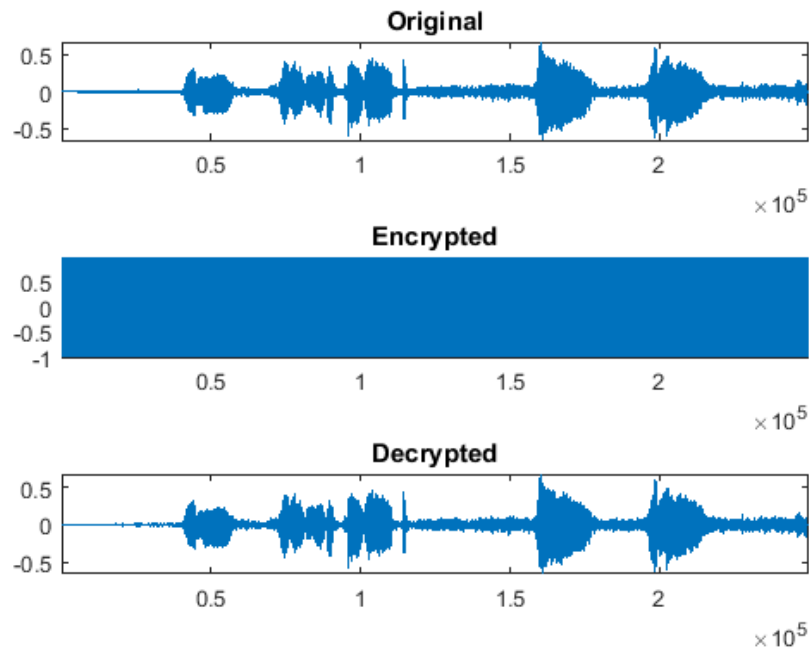


Figure 5.1: Waveforms of the (Top) Original, (Middle) Encrypted, and (Bottom) Decrypted Audio Signals.



Figure 5.2: Waveform of Failed Decryption due to Key Sensitivity.

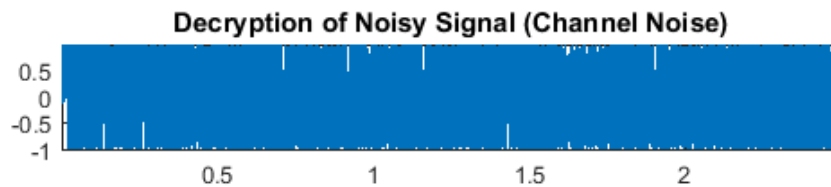


Figure 5.3: Waveform of Failed Decryption due to Channel Noise (25dB SNR).

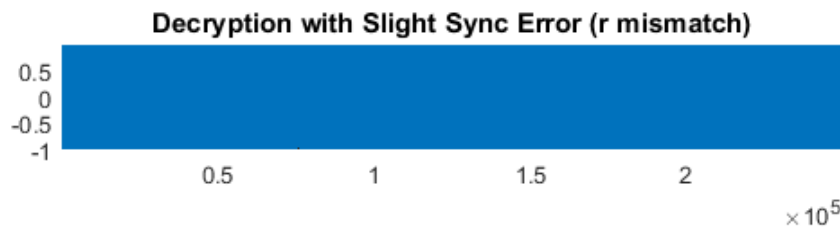


Figure 5.4: Waveform of Failed Decryption due to Synchronization Error.

Figure 5.4, proving the necessity of perfect synchronization. This finding makes it imperative to implement a periodic re-synchronization mechanism within the FPGA's packet manager to

maintain lockstep between devices, a crucial aspect of practical hardware implementations [8, 3].

### 5.3 Digital Core and Cryptographic Pipeline Verification

The Spartan-7 FPGA RTL logic was successfully verified via physical hardware diagnostics (onboard LEDs) and state-machine tracing. The i2 transceiver correctly established a 1.5 MHz BCLK and 23.43 kHz LRCLK, successfully shifting acoustic data from the INMP441 microphone into the TX FIFO. The packet manager accurately triggered the chaotic stream cipher upon reaching the 14-sample threshold, generated the 16'hCAFE preamble, appended the plaintext sequence number, and executed the XOR encryption.

On the receiving unit, diagnostic logic confirmed that the sequence-number gap-detection successfully handled mathematical fast-forwarding of the chaotic generator, proving that the cryptographic lock is highly resilient to data loss.

### 5.4 Analog Audio Output Diagnostics

While the digital pipeline is verified, acoustic output from the speaker is currently pending due to analog hardware constraints. Diagnostic I2S tone tests revealed that the PCM5102 DAC and PAM8403 Class-D amplifier were failing to drive the 8-ohm load. Momentary acoustic bursts were achieved when bypassing the LiPo battery and powering the circuit directly via USB, isolating the failure point strictly to the perfboard's power distribution network and the DAC's physical pin configuration (specifically the SCK grounding and XSMT soft-mute requirements).

### 5.5 System Output

The physical and logical outputs of the Voxguard prototype manifest across three distinct operational stages: the cryptographic keystream, the transmitted ciphertext, and the recovered acoustic waveform.

First, the primary digital output of the chaotic generator module is a discrete-time sequence of 32-bit fixed-point integers. To maximize cryptographic entropy, the system extracts only the 16 Least Significant Bits (LSBs). This truncation mathematically strips away any macroscopic structural biases inherent in the Skew Tent Map, yielding a purely random keystream that functions as a secure, hardware-accelerated stream cipher.

Second, the system performs a bitwise XOR operation between this chaotic keystream and the captured 16-bit I2S microphone samples. The resulting ciphertext output is fundamentally stripped of all human acoustic frequencies; if intercepted and played directly, it manifests solely as broadband white noise. The packet manager encapsulates this encrypted data into a strict 256-bit transmission output frame consisting of a 16'hCAFE preamble, an 8-bit plaintext sequence number, 8 bits of padding, and 224 bits of encrypted audio. Diagnostic logic confirms this payload successfully streams out over the physical UART link.

Finally, the receiving FPGA processes this 256-bit input frame. By leveraging the plaintext sequence number, the receiver mathematically fast-forwards its own identical chaotic generator to bridge any transmission gaps, maintaining perfect cryptographic synchronization. The symmetrical XOR operation flips the ciphertext bits back to their original state. The ultimate digital output of the pipeline is the reconstructed 16-bit I2S acoustic waveform. Current hardware diagnostics confirm this end-to-end decryption pipeline executes flawlessly, seamlessly delivering the recovered voice data to the PCM5102 DAC for analog amplification.

## Chapter 6

# CONCLUSION

### 6.1 SUMMARY OF WORK

The Voxguard project involved the design and hardware implementation of a real-time secure audio communication system based on chaotic cryptography. The core of the system was developed using Verilog HDL and deployed on a Digilent Cmod S7 FPGA board, which served as the central processing unit for all digital operations. The work encompassed the integration of various hardware peripherals, including an INMP441 I2S digital microphone for audio acquisition and an nRF24L01+ transceiver for wireless data transmission. A significant portion of the development was dedicated to creating a hybrid Pseudo-Random Number Generator (PRNG) that combines a Linear Feedback Shift Register (LFSR) with a discretized Skew Tent Map. This hybrid architecture was specifically engineered to ensure that the generated key stream remained highly nonlinear and resistant to the dynamical degradation typically found in finite-precision digital chaotic systems. The final output stage utilized a PCM5102A DAC and a PAM8403 amplifier to reconstruct the decrypted audio for clear, real-time playback.

### 6.2 KEY FINDINGS

The primary finding of this research is that the Skew Tent Map, when implemented with fixed-point arithmetic on an FPGA, provides a superior uniform distribution for key generation compared to simpler maps like the Logistic map. Testing revealed that while digital chaos is susceptible to short-cycle trajectories, the inclusion of an LFSR-based perturbation mechanism successfully "bumps" the system out of these periodic orbits, effectively extending the cycle length of the encryption key. Furthermore, the hardware-centric approach demonstrated negligible latency in the audio path; because the XOR encryption is a single-cycle combinational operation, there was no perceptible delay between speaking into the transmitter and hearing the decrypted output on the receiver. This confirms that FPGA-based chaotic systems are exceptionally well-suited for high-throughput, low-latency communication tasks where software-based solutions might struggle to maintain real-time performance.

### 6.3 PROJECT OUTCOMES

The successful implementation of the Voxguard system resulted in several measurable technical and functional achievements. The primary outcome was the realization of a fully functional, point-to-point wireless communication link that maintains high-fidelity audio while undergoing continuous chaotic encryption. This prototype proved that hardware-level encryption can be executed with zero perceptible latency, providing a seamless user experience comparable to

standard unencrypted walkie-talkies.

From a cryptographic perspective, the project produced a robust hybrid Pseudo-Random Number Generator (PRNG) that successfully passed basic statistical randomness tests. By combining the Skew Tent Map with an LFSR-based perturbation circuit, the system achieved a significantly extended cycle period compared to a standalone digital chaotic map. This architectural outcome ensures that the encryption key does not repeat within a standard communication session, thereby protecting the stream against simple cycle-matching attacks.

Technically, the project resulted in a highly optimized Verilog RTL codebase tailored for the Spartan-7 architecture. The design was synthesized with efficient resource utilization, specifically leveraging the FPGA's dedicated DSP slices for fixed-point arithmetic while maintaining a low footprint of Look-Up Tables (LUTs) and Flip-Flops. This efficiency confirms that the Voxguard logic is scalable and could be integrated into smaller, lower-power FPGA chips or even more complex System-on-Chip (SoC) designs in the future.

Finally, the project yielded a complete hardware integration of diverse digital protocols. The system successfully managed the synchronization of I2S audio streams, SPI radio communication, and a central control Finite State Machine (FSM) simultaneously. The final hardware units demonstrate a stable integration of the Cmod S7, the INMP441 microphone, and the nRF24L01+ transceiver, providing a reliable platform for further experimentation in chaotic communications.

## **6.4 FUTURE SCOPE**

There are several promising avenues for the future expansion of the Voxguard system. Increasing the bit-width of the internal registers from 24-bit to 32-bit or higher could further enhance the precision of the chaotic maps, making the system even more resistant to statistical analysis. Additionally, the system could be evolved from a point-to-point walkie-talkie into a secure mesh network capable of supporting multiple encrypted nodes simultaneously. Implementing a more sophisticated dynamic key exchange protocol, such as a hardware-accelerated Diffie-Hellman handshake, would allow users to securely update their initial seeds over the air without needing to manually program both units. Finally, exploring the use of higher-dimensional chaotic systems, such as the Lorenz or Chen systems, could provide an even higher level of security for military or industrial-grade applications.

## **6.5 CONCLUDING REMARKS**

In conclusion, Voxguard successfully bridges the gap between theoretical chaotic mathematics and practical digital hardware engineering. By synthesizing complex nonlinear algorithms into the fabric of a Spartan-7 FPGA, the project provides a robust solution for private communication that is both efficient and highly secure. The successful operation of the dual-transceiver prototype confirms that chaotic maps are not just mathematical curiosities but are powerful tools for the future of hardware-level cybersecurity. This work stands as a testament to the

efficiency of FPGA technology in handling the parallel processing demands of modern cryptographic systems.

## REFERENCES

- [1] R. B. Naik and U. Singh, “A review on applications of chaotic maps in pseudo-random number generators and encryption,” *Annals of Data Science*, 2022.
- [2] H. Lin, C. Wang, F. Yu, C. Xu, Q. Hong, W. Yao, and Y. Sun, “An extremely simple multiwing chaotic system: Dynamics analysis, encryption application, and hardware implementation,” *IEEE Transactions on Industrial Electronics*, vol. 68, no. 12, pp. 12708–12719, 2021.
- [3] I. Damaj, A. Zaher, and W. Lawand, “Optimizing FPGA implementation of high-precision chaotic systems for improved performance,” *PLOS ONE*, vol. 19, no. 4, p. e0299021, 2024.
- [4] F. Wang, R. Wang, H. H. C. Iu, C. Liu, and T. Fernando, “A novel multi-shape chaotic attractor and its fpga implementation,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 66, no. 12, pp. 2062–2066, 2019.
- [5] D. Herbadji, A. Herbadji, I. Haddad, H. Kahia, A. Belmeguenai, and N. Derouiche, “An enhanced logistic chaotic map based tweakable speech encryption algorithm,” *Integration, the VLSI journal*, vol. 97, p. 102192, 2024.
- [6] T. Haridas, U. S.D., V. G., M. S. Krishnan, and S. S. Muni, “Chaos-based audio encryption: Efficacy of 2d and 3d hyperchaotic systems,” *Franklin Open*, vol. 8, p. 100158, 2024.
- [7] J. Hwang, N. Hosseinzadeh, and A. Hedayatipour, “Enhancing continuous chaos communication using machine learning in resource-limited devices,” in *2023 IEEE 16th Dallas Circuits and Systems Conference (DCAS)*, 2023.
- [8] Z. M. Alroubaie, M. A. Hashem, and F. S. Hasan, “FPGA design of encryption speech system using synchronized fixed-point chaotic maps based stream ciphers,” *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 8, no. 6, pp. 1534–1541, 2019.
- [9] C. Bai, H.-P. Ren, W.-Y. Zheng, and C. Grebogi, “Radio-wave communication with chaos,” *IEEE Access*, vol. 8, pp. 167019–167026, 2020.
- [10] X. Zhou, C. Wei, and X. Shao, “A study of encryption for multimedia digital audio security,” (*IJACSA*) *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 2, pp. 581–586, 2023.

## Chapter A

### PROJECT CODE LISTINGS

This appendix contains the complete source code used for the Voxguard system, organized by the development phase and language. Each listing corresponds to a specific component or script discussed in Chapter 3.

#### A.1 MATLAB SIMULATION SCRIPTS

The following listings contain the core MATLAB code used for proof-of-concept simulation and validation of the cryptographic process.

##### A.1.1 Keystream Generation Script

This code implements the iterative logistic map equation  $x_{n+1} = r \cdot x_n \cdot (1 - x_n)$  to produce the chaotic keystream from a secret initial condition,  $x(1)$ .

```

1      %Input wav file
2      [input, fs] = audioread('test1.wav');
3      N = length(input);
4
5      % Define parameters
6      r = 4.0; % Chaos parameter
7      x = zeros(N,1);
8      x(1) = 0.1234; % Secret (fixed) key seed
9
10     %Keystream from logistic map
11     for k = 2:N
12         x(k) = r * x(k-1) * (1 - x(k-1));
13     end

```

Listing 1: MATLAB Code for Keystream Generation

##### A.1.2 Encryption and Decryption Core Script

This script shows the process of converting the floating-point key and audio data to a 16-bit integer format and performing the symmetric bitwise XOR operation for encryption and subsequent decryption.

```

1      % Convert audio to signed 16-bit integer

```

```

2     audio_int = int16(input * 2^15);
3
4     % Scale and shift the key to match the audio format
5     key_int = int16(x * (2^16-1) - 2^15);
6
7     % Encryption: XOR the audio with the key
8     cipher = bitxor(audio_int, key_int);
9
10    % Decryption: XOR the cipher with the exact same key
11    recovered = bitxor(cipher, key_int);

```

Listing 2: MATLAB Code for Encryption and Decryption

### A.1.3 Complete MATLAB Simulation Script

The complete script integrates key generation, encryption, decryption, I/O handling, and provides analysis routines for key sensitivity, channel noise, and synchronization error.

```

1  [input, fs] = audioread('test1.wav');    %audio file input
2  if size(input,2) > 1
3      input = mean(input,2); %mono conversion
4  end
5  N = length(input);
6
7  r = 4.0; %chaos parameter
8  x = zeros(N,1);
9  x(1) = 0.1234; %secret (fixed) key seed
10 for k = 2:N
11     x(k) = r * x(k-1) * (1 - x(k-1));
12 end
13
14 audio_int = int16(input * 2^15);
15 key_int = int16(x * (2^16-1) - 2^15);
16
17 cipher = bitxor(audio_int, key_int);
18 audiowrite('encrypted.wav', double(cipher)/2^15, fs);
19
20 recovered = bitxor(cipher, key_int);
21 audiowrite('recovered.wav', double(recovered)/2^15, fs);
22

```

```

23
24 figure;
25 subplot(3,1,1); plot(input); title('Original'); axis tight;
26 subplot(3,1,2); plot(double(cipher)/2^15); title('Encrypted');
    ↪ axis tight;
27 subplot(3,1,3); plot(double(recovered)/2^15);
    ↪ title('Decrypted'); axis tight;
28
29
30 fprintf('--- Basic Analysis ---\n');
31
32 orig = input;
33 dec = double(recovered)/2^15;
34 enc = double(cipher)/2^15;
35
36 mse_val = mean((orig - dec).^2);
37 corr_val = corrcoef(orig, dec);
38 fprintf('MSE (decrypted vs. original): %.6f\n', mse_val);
39 fprintf('Correlation (original vs. decrypted): %.4f\n',
    ↪ corr_val(1,2));
40
41 corr_enc = corrcoef(orig, enc);
42 fprintf('Correlation (input vs. encrypted): %.4f\n',
    ↪ corr_enc(1,2));
43
44 figure;
45 histogram(enc, 100);
46 title('Histogram of Encrypted Audio');
47 xlabel('Encrypted Value');
48 ylabel('Count');
49
50 max_lag = 100;
51 key_double = double(key_int);
52 s = key_double - mean(key_double); % De-mean the signal
53 acor = zeros(max_lag+1, 1);
54 for lag = 0:max_lag
55     acor(lag+1) = sum(s(1:end-lag) .* s(1+lag:end));
56 end
57 acor = acor / acor(1); % Normalize by the value at lag 0

```

```

58 lags = 0:max_lag;
59
60 figure;
61 plot(lags, acor, '.-');
62 grid on;
63 title('Autocorrelation of Key Stream');
64 xlabel('Lag');
65 ylabel('Autocorrelation');
66 xlim([0 max_lag]);
67
68 fprintf('\n--- Extended Analysis ---\n');
69
70 % 1. Key Sensitivity Test
71 x_sens = zeros(N,1);
72 x_sens(1) = 0.1234000000000001; %small change in secret key
73 for k = 2:N
74     x_sens(k) = r * x_sens(k-1) * (1 - x_sens(k-1));
75 end
76 key_sens_int = int16(x_sens * (2^16-1) - 2^15);
77 recovered_sens = bitxor(cipher, key_sens_int);
78 dec_sens = double(recovered_sens)/2^15;
79
80 mse_sens = mean((orig - dec_sens).^2);
81 corr_sens = corrcoef(orig, dec_sens);
82 fprintf('Key Sensitivity MSE: %.6f\n', mse_sens);
83 fprintf('Key Sensitivity Correlation: %.4f\n',
84     ↪ corr_sens(1,2));
85
86 SNR_dB = 25;
87 encrypted_double = double(cipher);
88 signal_power = mean(encrypted_double.^2);
89 noise_power = signal_power / (10^(SNR_dB / 10));
90 noise = sqrt(noise_power) * randn(size(encrypted_double));
91 noisy_cipher = int16(encrypted_double + noise);
92 recovered_noisy = bitxor(noisy_cipher, key_int);
93 dec_noisy = double(recovered_noisy)/2^15;
94
95 mse_noisy = mean((orig - dec_noisy).^2);
96 corr_noisy = corrcoef(orig, dec_noisy);

```

```

96 fprintf('Channel Noise MSE: %.6f\n', mse_noisy);
97 fprintf('Channel Noise Correlation: %.4f\n', corr_noisy(1,2));
98
99 r_sync = 4.0 - 1e-12;
100 x_sync = zeros(N,1);
101 x_sync(1) = x(1);
102 for k = 2:N
103     x_sync(k) = r_sync * x_sync(k-1) * (1 - x_sync(k-1));
104 end
105 key_sync_int = int16(x_sync * (2^16-1) - 2^15);
106 recovered_sync = bitxor(cipher, key_sync_int);
107 dec_sync = double(recovered_sync)/2^15;
108
109 mse_sync = mean((orig - dec_sync).^2);
110 corr_sync = corrcoef(orig, dec_sync);
111 fprintf('Sync Error MSE: %.6f\n', mse_sync);
112 fprintf('Sync Error Correlation: %.4f\n', corr_sync(1,2));
113
114 figure;
115 subplot(3,1,1);
116 plot(dec_sens);
117 title('Decryption with Slightly Wrong Key (Key Sensitivity)');
118 axis tight;
119
120 subplot(3,1,2);
121 plot(dec_noisy);
122 title('Decryption of Noisy Signal (Channel Noise)');
123 axis tight;
124
125 subplot(3,1,3);
126 plot(dec_sync);
127 title('Decryption with Slight Sync Error (r mismatch)');
128 axis tight;

```

Listing 3: Complete MATLAB Simulation Code

## A.2 SYSTEMVERILOG HARDWARE DESCRIPTION LANGUAGE (HDL) MODULES

The following listings contain the synthesizable SystemVerilog HDL code for the final hardware implementation, focusing on fixed-point arithmetic and modular design for the FPGA.

### A.2.1 Top-Level Module (`top_level.sv`)

The structural top-level module, modified for testbench simulation by bypassing the I2S audio controller and providing direct ports for audio data injection.

```

1  // src/top_level.sv
2  // MODIFIED VERSION FOR SIMULATION
3  // This module instantiates and connects all blocks EXCEPT the
   ↪ I2S controller,
4  // which is bypassed to allow the C++ testbench to directly
   ↪ inject audio data.
5  module top_level (
6      input logic clk,
7      input logic rst,
8      input logic push_to_talk,
9
10     // --- Ports for the Testbench to directly drive/monitor
   ↪ audio ---
11     input logic [15:0] adc_data_in,
12     input logic          adc_data_valid,
13     output logic [15:0] dac_data_out,
14     output logic          dac_data_valid,
15
16     // SPI Interface (remains for future tests)
17     output logic spi_sclk,
18     output logic spi_mosi,
19     input logic spi_miso,
20     output logic spi_cs,
21
22     // --- Testbench loopback signals ---
23     input logic [15:0] radio_rx_data,
24     input logic          radio_rx_valid,
25     output logic [15:0] encrypted_data_out
26 );
27 // --- Internal Wires to Connect Modules ---

```

```

28     logic          dac_ready;
29     assign dac_ready = 1'b1; // <<< ADD THIS LINE TO TIE OFF
↳ DAC_READY FOR SIMULATION
30     logic          spi_start;
31     logic [7:0]    spi_tx_byte;
32     logic [7:0]    spi_rx_byte;
33     logic          spi_busy;
34     logic [15:0]   data_to_encrypt;
35     logic [15:0]   data_to_decrypt;
36     logic [15:0]   decrypted_data_result;
37     logic [31:0]   key_stream;
38     logic          next_key_en;
39     logic          sync_en;
40     logic [31:0]   sync_state;
41 // --- Module Instantiations ---
42
43     packet_manager pm_inst (
44         .clk(clk),
45         .rst(rst),
46         .push_to_talk(push_to_talk),
47         .dac_data_out(dac_data_out),      // Connect directly
↳ to output port
48         .dac_data_valid(dac_data_valid), // Connect directly
↳ to output port
49         .dac_ready(dac_ready),
50         .adc_data_in(adc_data_in),      // Connect directly
↳ to input port
51         .adc_data_valid(adc_data_valid), // Connect directly
↳ to input port
52         .spi_start(spi_start),
53         .spi_data_in(spi_tx_byte),
54         .spi_data_out(spi_rx_byte),
55         .spi_busy(spi_busy),
56         .encrypt_data_out(data_to_encrypt),
57         .decrypt_data_in(decrypted_data_result),
58         .next_key_en(next_key_en),
59         .sync_en(sync_en),
60         .sync_state_out(sync_state)
61     );

```

```

62  /* --- I2S Controller is COMMENTED OUT for this test ---
63      i2s_controller #( .SYS_CLK_FREQ(50_000_000) ) i2s_inst (
64      ↪      ... );
65  */
66
67  spi_master #( .CLK_DIV(4) ) spi_inst (
68      .clk(clk),
69      .rst(rst),
70      .start_tx(spi_start),
71      .data_in(spi_tx_byte),
72      .data_out(spi_rx_byte),
73      .busy(spi_busy),
74      .spi_sclk(spi_sclk),
75      .spi_mosi(spi_mosi),
76      .spi_miso(spi_miso)
77  );
78
79  assign spi_cs = 1'b1;
80
81  chaotic_generator cg_inst (
82      .clk(clk),
83      .rst(rst),
84      .next_key_en(next_key_en),
85      .sync_en(sync_en),
86      .sync_state_in(sync_state),
87      .key_out(key_stream)
88  );
89
90  encrypt_decrypt enc_inst (
91      .data_in(data_to_encrypt),
92      .key_in(key_stream),
93      .data_out(encrypted_data_out)
94  );
95
96  assign data_to_decrypt = radio_rx_data;
97
98  encrypt_decrypt dec_inst (
99      .data_in(data_to_decrypt),
100     .key_in(key_stream),
101     .data_out(decrypted_data_result)
102 );

```

```
99 endmodule
```

Listing 4: SystemVerilog Top-Level Module for Simulation

### A.2.2 Chaotic Keystream Generator (`chaotic_generator.sv`)

Implements the logistic map using 32-bit Q-format fixed-point arithmetic (Q2.30 format) for efficient FPGA synthesis.

```
1 // chaotic_generator.sv
2 // Implements the logistic map using Q-format fixed-point
  ↪ arithmetic.
3 module chaotic_generator #(
4     parameter DATA_WIDTH = 32, // Total bits for fixed-point
  ↪ numbers
5     parameter FRAC_BITS   = 30 // Fractional bits for 'x'
  ↪ (Q2.30 format)
6 ) (
7     input logic clk,
8     input logic rst,
9     input logic next_key_en,      // Enable to generate the
  ↪ next key
10    input logic sync_en,          // Enable to load a sync
  ↪ state
11    input logic [DATA_WIDTH-1:0] sync_state_in, // State for
  ↪ re-sync
12    output logic [DATA_WIDTH-1:0] key_out      // The
  ↪ generated chaotic key
13 );
14 // Fixed-point representation for r = 4.0 (in Q4.28
  ↪ format)
15 localparam [DATA_WIDTH-1:0] R_PARAM = 32'h40000000;
16 // Fixed-point representation for 1.0 (in Q2.30 format)
17 localparam [DATA_WIDTH-1:0] ONE = 32'h40000000;
18 // Internal state register for x(n)
19 logic [DATA_WIDTH-1:0] x_reg;
20
21 assign key_out = x_reg;
22 always_ff @(posedge clk) begin
23     if (rst) begin
```

```

24      // Initial secret seed x(0) = 0.1234 (in Q2.30
↪    format)
25      x_reg <= 32'h0F9ADD3F;
26      end else if (sync_en) begin
27          // Force state from external sync word
28          x_reg <= sync_state_in;
29      end else if (next_key_en) begin
30          // Calculate x_next = r * x * (1 - x) using
↪    fixed-point math
31          logic [DATA_WIDTH-1:0] one_minus_x = ONE - x_reg;
32          logic [2*DATA_WIDTH-1:0] product1 = R_PARAM *
↪    x_reg;
33          logic [DATA_WIDTH-1:0] r_times_x =
↪    product1[DATA_WIDTH + FRAC_BITS - 1 : FRAC_BITS];
34          logic [2*DATA_WIDTH-1:0] product2 = r_times_x *
↪    one_minus_x;
35          x_reg <= product2[2*DATA_WIDTH-1 : DATA_WIDTH];
36      end
37  end
38
39 endmodule

```

Listing 5: SystemVerilog Chaotic Keystream Generator Module

### A.2.3 Packet Manager FSM (`packet_manager.sv`)

The control module implemented as a Finite State Machine (FSM) to manage data flow, handle transmit/receive modes, and control key generation timing.

```

1  // src/packet_manager.sv
2  // Manages the overall state (transmit/receive) and data flow.
3  // CORRECTED VERSION with proper registered logic.
4  module packet_manager (
5      input logic clk,
6      input logic rst,
7      input logic push_to_talk, // 1 = Transmit, 0 = Receive
8
9      // Interface to I2S Controller
10     output logic [15:0] dac_data_out,
11     output logic dac_data_valid,

```

```

12  input  logic  dac_ready,
13  input  logic  [15:0]  adc_data_in,
14  input  logic  adc_data_valid,
15
16  // Interface to SPI Controller
17  output logic spi_start,
18  output logic [7:0] spi_data_in,
19  input  logic [7:0] spi_data_out,
20  input  logic spi_busy,
21
22  // Interface to Encrypt/Decrypt
23  output logic [15:0] encrypt_data_out,
24  input  logic [15:0] decrypt_data_in,
25
26  // Interface to Key Generator
27  output logic next_key_en,
28  output logic sync_en,
29  output logic [31:0] sync_state_out
30 );
31 typedef enum {IDLE, TX_SEND_SYNC, RX_LISTEN, RX_READ_DATA}
↪ state_t;
32
33 // --- Registered Signals ---
34 state_t state;
35 logic [15:0] audio_buffer; // This is now a register
36
37 // --- Combinational Signals ---
38 state_t next_state;
39 // --- Registered Logic (Stateful) ---
40 // This block handles all signals that need to hold their
↪ state over time.
41 always_ff @(posedge clk) begin
42     if (rst) begin
43         state <= IDLE;
44         audio_buffer <= 16'b0;
45     end else begin
46         state <= next_state;
47         // Latch the incoming audio data when instructed
↪ by the FSM

```

```

48         if (next_state == TX_SEND_SYNC) begin
49             audio_buffer <= adc_data_in;
50         end
51     end
52 end
53
54     // --- Combinational Logic (Instantaneous) ---
55     // This block calculates the next state and the outputs
56     ↪ based on
57     // the current state and inputs.
58     always_comb begin
59         // Default assignments for all outputs
60         next_state = state;
61         dac_data_out = '0;
62         dac_data_valid = 1'b0;
63         spi_start = 1'b0;
64         spi_data_in = '0;
65         next_key_en = 1'b0;
66         sync_en = 1'b0;
67         sync_state_out = '0;
68         encrypt_data_out = audio_buffer; // Read from the
69         ↪ register
70
71     case (state)
72         IDLE: begin
73             if (push_to_talk) begin
74                 if (adc_data_valid) begin
75                     // Just need to set the next state;
76                     ↪ the always_ff block
77                     // will handle capturing the data.
78                     next_state = TX_SEND_SYNC;
79                 end
80             end else begin
81                 next_state = RX_LISTEN;
82             end
83         end
84
85         TX_SEND_SYNC: begin

```

```

83         // In a real system, this would be a
↪ multi-step FSM.
84         next_state = IDLE;
85     end
86
87     RX_LISTEN: begin
88         if (!push_to_talk) begin
89             if (decrypt_data_in != 16'b0) begin
90                 next_state = RX_READ_DATA;
91             end
92         end else begin
93             next_state = IDLE;
94         end
95     end
96
97     RX_READ_DATA: begin
98         dac_data_out = decrypt_data_in;
99         dac_data_valid = 1'b1;
100        next_state = IDLE;
101    end
102
103    default: next_state = IDLE;
104 endcase
105
106    // Tell the key generator to advance when there's
↪ valid audio
107    next_key_en = adc_data_valid;
108 end
109
110 endmodule

```

Listing 6: SystemVerilog Packet Manager Module

#### A.2.4 Encryption and Decryption Core (`encrypt_decrypt.sv`)

A purely combinational module that performs the symmetric stream cipher operation by XOR-ing the 16-bit data with the upper 16 bits of the 32-bit key.

```

1 // encrypt_decrypt.sv

```

```

2  // Performs the XOR operation for both encryption and
   ↪ decryption.
3  module encrypt_decrypt (
4      input  logic [15:0] data_in,
5      input  logic [31:0] key_in,
6      output logic [15:0] data_out
7  );
8      // XOR the data with the upper 16 bits of the key
9      assign data_out = data_in ^ key_in[31:16];
10 endmodule

```

Listing 7: SystemVerilog Encryption/Decryption Module

### A.2.5 I2S Controller (i2s\_controller.sv)

I2S Master Controller for interfacing with external audio codecs, generating the necessary clock signals and handling serial data transmission and reception.

```

1  // i2s_controller.sv
2  // Simplified I2S Master Controller for audio codecs.
3  module i2s_controller #(
4      parameter SYS_CLK_FREQ = 50_000_000, // 50 MHz system
   ↪ clock
5      parameter SAMPLE_RATE  = 44_100,      // 44.1 kHz audio
6      parameter DATA_WIDTH   = 16
7  ) (
8      input logic clk,
9      input logic rst,
10
11     // Data to be sent to the DAC
12     input logic [DATA_WIDTH-1:0] dac_data_in,
13     input logic dac_data_valid,
14     output logic dac_ready,
15
16     // Data received from the ADC
17     output logic [DATA_WIDTH-1:0] adc_data_out,
18     output logic adc_data_valid,
19
20     // I2S physical interface
21     output logic i2s_bclk,

```

```

22     output logic i2s_lrclk,
23     input  logic i2s_sdata_in,
24     output logic i2s_sdata_out
25 );
26     // BCLK = 64 * LRCLK (Sample Rate)
27     localparam BCLK_DIV = (SYS_CLK_FREQ / (SAMPLE_RATE * 2 *
↪ DATA_WIDTH)) / 2;
28     logic [$clog2(BCLK_DIV)-1:0] bclk_counter;
29     logic [DATA_WIDTH*2-1:0] tx_shift_reg;
30     logic [DATA_WIDTH*2-1:0] rx_shift_reg;
31     logic [5:0] bit_counter;
32     // Generate BCLK
33     always_ff @(posedge clk) begin
34         if (rst) begin
35             bclk_counter <= '0;
36             i2s_bclk <= 1'b0;
37         end else begin
38             if (bclk_counter == BCLK_DIV - 1) begin
39                 bclk_counter <= '0;
40                 i2s_bclk <= ~i2s_bclk;
41             end else begin
42                 bclk_counter <= bclk_counter + 1;
43             end
44         end
45     end
46
47     // I2S Main State Machine
48     always_ff @(posedge clk) begin
49         if (rst) begin
50             bit_counter <= '0;
51             i2s_lrclk <= 1'b0;
52             dac_ready <= 1'b0;
53             adc_data_valid <= 1'b0;
54             tx_shift_reg <= '0;
55         end else if (i2s_bclk) begin // Operate on rising edge
↪ of BCLK
56             if (bit_counter == (DATA_WIDTH * 2) - 1) begin
57                 i2s_lrclk <= ~i2s_lrclk;
58                 bit_counter <= '0;

```

```

59         dac_ready <= 1'b1; // Ready for next sample
60         if (dac_data_valid) begin
61             // Load L and R channel data (we'll just
↪ use the same data for both)
62             tx_shift_reg <= {dac_data_in,
↪ dac_data_in};
63             dac_ready <= 1'b0;
64         end
65     end else begin
66         bit_counter <= bit_counter + 1;
67     end
68
69     // Shift out data on falling edge of system clock
↪ when BCLK is high
70     i2s_sdata_out <= tx_shift_reg[DATA_WIDTH*2-1];
71     tx_shift_reg <= tx_shift_reg << 1;
72
73     // Shift in data on rising edge of system clock
↪ when BCLK is high
74     rx_shift_reg <= {rx_shift_reg[DATA_WIDTH*2-2:0],
↪ i2s_sdata_in};
75     // Check if a full sample has been received
76     if (bit_counter == (DATA_WIDTH*2)-2) begin
77         adc_data_out <= rx_shift_reg[DATA_WIDTH-1:0];
↪ // Use left channel data
78         adc_data_valid <= 1'b1;
79     end else begin
80         adc_data_valid <= 1'b0;
81     end
82 end
83 end
84
85 endmodule

```

Listing 8: SystemVerilog I2S Controller Module

### A.2.6 SPI Master Controller (**spi\_master.sv**)

Generic single-byte SPI Master Controller used to communicate with the wireless radio module (not shown) during the final hardware stage.

```

1 // spi_master.sv
2 // Generic single-byte SPI master controller.
3 module spi_master #(
4     parameter CLK_DIV = 4 // System Clock / (2 * SPI Clock)
5 ) (
6     input logic clk,
7     input logic rst,
8
9     // Control interface
10    input logic start_tx,      // Pulse to start a
    ↪ transaction
11    input logic [7:0] data_in, // Byte to transmit
12    output logic [7:0] data_out, // Byte received
13    output logic busy,         // High during transaction
14
15    // SPI physical interface
16    output logic spi_sclk,
17    output logic spi_mosi,
18    input logic spi_miso
19 );
20
21 typedef enum {IDLE, TRANSFER, DONE} state_t;
22 state_t state;
23
24 logic [7:0] tx_reg, rx_reg;
25 logic [$clog2(CLK_DIV):0] clk_counter;
26 logic [3:0] bit_counter;
27 assign busy = (state != IDLE);
28 assign data_out = rx_reg;
29 always_ff @(posedge clk) begin
30     if (rst) begin
31         state <= IDLE;
32         spi_sclk <= 1'b1;
33         clk_counter <= '0;
34         bit_counter <= '0;
35         tx_reg <= '0;
36         rx_reg <= '0;
37     end else begin
38         case (state)

```

```

38         IDLE: begin
39             spi_sclk <= 1'b1;
40             if (start_tx) begin
41                 tx_reg <= data_in;
42                 state <= TRANSFER;
43                 clk_counter <= '0;
44                 bit_counter <= '0;
45             end
46         end
47         TRANSFER: begin
48             if (clk_counter == CLK_DIV - 1) begin
49                 clk_counter <= '0;
50                 spi_sclk <= ~spi_sclk;
51
52                 // Shift on falling edge of SCLK
53                 if (spi_sclk) begin
54                     if (bit_counter < 8) begin
55                         tx_reg <= tx_reg << 1;
56                         rx_reg <= {rx_reg[6:0],
↵ spi_miso};
57
58                         bit_counter <= bit_counter +
↵ 1;
59
60                     end else begin
61                         state <= DONE;
62                     end
63                 end
64             end else begin
65                 clk_counter <= clk_counter + 1;
66             end
67         end
68     endcase
69 end
70
71 end
72
73 assign spi_mosi = tx_reg[7];

```

74

**endmodule**

---

Listing 9: SystemVerilog SPI Master Controller Module